

**Linux Privilege Escalation Automation Toolkit:  
Design, Implementation, and Security Analysis of an Automated  
Vulnerability Scanner for Linux Systems**



**Nitesh Ghimire**

**BScIT, APU**

**Submitted to: unifiedmentor**

**April 12, 2026**

## **Abstract**

This report presents the design, development, and evaluation of the Linux Privilege Escalation Automation Toolkit (PRIVESC), a modular, detection-only security scanner built entirely in Python 3 without external dependencies. The toolkit automates the identification of common privilege escalation vectors on Linux-based systems, including misconfigurations related to SUID/SGID binaries, insecure file permissions, vulnerable service definitions, exploitable cron jobs, outdated kernel versions with known Common Vulnerabilities and Exposures (CVE) identifiers, dangerous Linux capability assignments, exposed credentials, and PATH hijacking opportunities. The system comprises eight independent scanner modules, a weighted risk-scoring engine, and a multi-format report generator capable of producing text, JSON, and self-contained HTML output. A live evaluation was conducted on a Kali GNU/Linux 2025.2 system running kernel version 6.12.25-amd64. The scan identified 114 total findings distributed across five categories, with six critical-severity and five high-severity issues. The most significant findings involved hardcoded credentials in world-readable configuration files and missing service binaries susceptible to binary planting. Results demonstrate that automated scanning can efficiently surface real misconfigurations that manual auditing may overlook. The project adheres to a strict detection-only design philosophy and includes comprehensive ethical use documentation.

**Keywords:** privilege escalation, Linux security, vulnerability scanner, penetration testing, SUID, GTFOBins, kernel CVE, credential exposure, PATH hijacking, security automation

## Table of Contents

|                                                                  |    |
|------------------------------------------------------------------|----|
| Introduction.....                                                | 5  |
| Background and Related Work.....                                 | 5  |
| Privilege Escalation in Linux Systems.....                       | 5  |
| Existing Tools .....                                             | 6  |
| System Architecture and Design.....                              | 7  |
| Design Philosophy .....                                          | 7  |
| System Overview .....                                            | 7  |
| Project Structure.....                                           | 7  |
| Scanner Module Design.....                                       | 8  |
| Module 2a: SUID/SGID Binary Scanner.....                         | 8  |
| Module 2b: File and Directory Permission Scanner .....           | 8  |
| Module 2c: Service and Sudo Scanner .....                        | 8  |
| Module 2d: Cron Job Scanner.....                                 | 8  |
| Module 2e: Kernel CVE Scanner.....                               | 9  |
| Module 2f: Linux Capabilities Scanner .....                      | 9  |
| Module 2g: Credential Scanner.....                               | 9  |
| Module 2h: PATH Hijacking Scanner.....                           | 10 |
| Analysis Engine .....                                            | 10 |
| Deduplication and Severity Normalisation.....                    | 10 |
| Risk Scoring.....                                                | 10 |
| Report Generation.....                                           | 11 |
| Text and JSON Reports.....                                       | 11 |
| HTML Report .....                                                | 11 |
| Implementation Results: Live Scan on Kali GNU/Linux 2025.2 ..... | 11 |

|                                                              |    |
|--------------------------------------------------------------|----|
| Test Environment.....                                        | 11 |
| Summary of Findings.....                                     | 12 |
| Critical Findings: Credential Exposure.....                  | 12 |
| High Findings.....                                           | 13 |
| Medium Findings: PATH Hijacking via Relative ExecStart ..... | 13 |
| Low Findings: SUID/SGID Binaries .....                       | 14 |
| Discussion.....                                              | 14 |
| Effectiveness of Automated Detection .....                   | 14 |
| False Positives and Limitations.....                         | 15 |
| Ethical Considerations .....                                 | 15 |
| Conclusion .....                                             | 16 |
| References.....                                              | 17 |

## Linux Privilege Escalation Automation Toolkit: Design, Implementation, and Security Analysis of an Automated Vulnerability Scanner for Linux Systems

### Introduction

Privilege escalation is one of the most consequential phases of a cyberattack. Once a threat actor obtains an initial foothold on a Linux system — even as an unprivileged user — a surprisingly wide range of configuration weaknesses, software vulnerabilities, and overlooked permissions can provide a pathway to root-level access (Andress & Winterfeld, 2014). The ability to identify these weaknesses before adversaries exploit them is a core capability of defensive security operations and authorised penetration testing.

Manual privilege escalation auditing is thorough but slow. A practitioner must inspect dozens of distinct attack surfaces — binary permission bits, scheduled tasks, kernel version history, capability flags, credential stores, and service configuration files — across a system that may contain thousands of files. Automation addresses this bottleneck. Tools such as LinPEAS (Privilege Escalation Awesome Scripts Suite) and LinEnum have demonstrated that scripted enumeration can surface critical misconfigurations in seconds (Polop, 2021). However, these tools are often written as complex shell scripts that are difficult to read, modify, or present in an educational context.

The Linux Privilege Escalation Automation Toolkit (PRIVESC) was developed to fill this gap. It is a clean, readable, fully modular Python 3 implementation of the most common privilege escalation checks, paired with a weighted risk-scoring engine and a multi-format report generator. The project is designed for three audiences: security students learning how privilege escalation works, system administrators hardening their own infrastructure, and authorised penetration testers conducting formal security engagements. The tool operates in a detection-only mode it identifies and reports vulnerabilities without exploiting any of them.

This report describes the system's architecture, the design rationale behind each module, the risk scoring methodology, the report generation pipeline, and the results of a live scan conducted on a Kali GNU/Linux 2025.2 system. It concludes with a discussion of findings, limitations, and directions for future development.

Background and Related Work

Privilege Escalation in Linux Systems

In Linux, the principle of least privilege dictates that processes and users should operate with only the permissions necessary for their function (Saltzer & Schroeder, 1975). When this principle is violated — through misconfiguration, software defects, or administrative oversight — an attacker with limited access can leverage the excess privilege to take full control of the system. The MITRE ATT&CK framework categorises privilege escalation as a distinct tactic (TA0004) and enumerates dozens of techniques applicable to Linux environments, including exploitation of SUID binaries (T1548.001), sudo misconfigurations (T1548.003), and cron job manipulation (T1053.003) (MITRE Corporation, 2023).

Key privilege escalation vectors on Linux include: (a) binaries with the Set User ID (SUID) bit set, which execute with the owner's privileges rather than the calling user's; (b) world-writable files or directories in sensitive locations; (c) sudo rules granting broad or passwordless command execution; (d) cron jobs that reference scripts or directories writable by non-root users; (e) running kernels with unpatched local privilege escalation vulnerabilities; (f) Linux capabilities assigned to binaries in ways that confer near-root power; (g) secrets stored in plaintext in configuration files or shell history; and (h) attacker-controlled directories appearing in the system PATH before trusted ones (Hacking Articles, 2021; Tipton & Krause, 2007).

### Existing Tools

Several open-source tools address automated Linux privilege escalation enumeration. LinPEAS, part of the PEASS-ng suite, is the most comprehensive, using a large bash script to enumerate hundreds of checks with colour-coded output (Polop, 2021). LinEnum provides a similar function in a more compact bash script (Rebootuser, 2019). BeRoot is a Python-based tool focused specifically on sudo and cron misconfigurations (Wagner, 2018). The GTF0Bins project (Wildner & Polop, 2019) maintains a curated, community-sourced database of Unix binaries that can be exploited through sudo, SUID bits, or capabilities to escape restricted environments or escalate privileges.

Existing tools prioritise comprehensiveness over readability and modularity. Shell scripts are difficult to extend, test in isolation, or adapt for structured output consumption. PRIVESC was designed to complement these tools by offering a Python-native, module-by-module architecture where each check is independently maintainable, testable, and understandable — making it as valuable for learning as it is for operational use.

## System Architecture and Design

### Design Philosophy

Three principles guided the design of PRIVESC. First, detection without exploitation: the tool reports vulnerabilities and provides contextual exploit examples for educational reference, but takes no automated action that modifies system state, creates persistence, or escalates privileges. Second, zero external dependencies: the entire codebase relies exclusively on the Python 3 standard library (subprocess, os, stat, re, json, glob, argparse, datetime), ensuring the tool can be deployed to any Python 3.6+ environment without installation friction. Third, graceful degradation: every module handles permission errors, missing system commands, and non-Linux environments without crashing, returning an empty findings list and optionally printing a diagnostic message.

### System Overview

PRIVESC follows a four-step pipeline orchestrated by `main.py`:

Step 1 — System information collection: baseline data (hostname, kernel release, current user, group memberships, shell users, sudo version, PATH) is collected and used to contextualise all subsequent findings.

Step 2 — Scanner modules: eight independent modules execute in sequence, each returning a list of structured finding dictionaries.

Step 3 — Analysis engine: all findings are aggregated, deduplicated, severity-normalised, sorted, and scored by a weighted composite algorithm.

Step 4 — Report generation: findings and summary statistics are rendered into one or more output formats (plain text, JSON, or HTML) and written to the `output/` directory.

The command-line interface supports selective module skipping (`--skip`), a quick scan mode (`--quick`) that bypasses slow filesystem-wide searches, verbose progress output (`--verbose`), and flexible format selection (`--format text|json|html|all`).

### Project Structure

The repository is organised into four top-level packages and a data directory, reflecting the four-step pipeline. The `modules/` package contains the eight scanner implementations. The `analysis/` package contains the risk scoring engine. The `reporter/` package contains text, JSON, and HTML report generators. The `data/` directory contains two reference databases: `gtfobins.json`, a curated set of 36 exploitable binaries with per-binary exploit commands and notes, and

kernel\_cves.json, a database of known local privilege escalation CVEs keyed by kernel version ranges.

## Scanner Module Design

### Module 2a: SUID/SGID Binary Scanner

The SUID (Set User ID) scanner uses the Linux `find` command to enumerate all files with the SUID (`-perm -4000`) or SGID (`-perm -2000`) bits set. In normal mode, the search covers the entire filesystem. In quick mode, it is limited to seven standard binary directories (`/usr/bin`, `/usr/sbin`, `/bin`, `/sbin`, `/usr/local/bin`, `/usr/local/sbin`, `/opt`), substantially reducing scan time. Each discovered binary is cross-referenced against `gtfobins.json` by filename. Matches are reported as HIGH severity findings with the GTFOBins exploit command embedded in the finding; unmatched SUID binaries are reported as LOW severity. A set of commonly benign SGID binaries (such as `wall`, `ssh-agent`, and `crontab`) is filtered to LOW severity unless a GTFOBins match is found.

### Module 2b: File and Directory Permission Scanner

The permission scanner checks two categories of issues. For critical system files — including `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`, and `/etc/crontab` — it inspects the permission octet using Python's `os.stat()` and reports any deviation from expected values (e.g., shadow files readable by non-root). For world-writable files and directories, it performs filesystem-wide find sweeps looking for paths with the other-writable bit set (`-perm -o+w`), filtering known safe temporary directories such as `/tmp` and `/proc`. In quick mode, the world-writable sweeps are skipped entirely and only critical file checks are performed.

### Module 2c: Service and Sudo Scanner

This module audits two distinct attack surfaces. For systemd services, it parses `/lib/systemd/system/` and `/etc/systemd/system/` service unit files, checking for `ExecStart` lines that reference binaries that do not exist on disk (binary planting risk) and for services running as root with world-writable script paths. For sudo policy, it executes `sudo -l` and parses the output for NOPASSWD grants, dangerous command allowances (such as unrestricted shell access), and wildcard rules that could be exploited for argument injection. Sudo checks are treated as HIGH severity given their direct path to root.

### Module 2d: Cron Job Scanner



The cron scanner enumerates scheduled tasks from `/etc/crontab`, all files in `/etc/cron.*` directories, and per-user crontab files. For each script referenced by a cron job running as root, it checks whether the script file is world-writable, whether it resides in a world-writable directory, and whether the script references non-existent binaries. Cron job vulnerabilities are among the most reliable privilege escalation techniques in CTF and real-world environments because they combine periodic root execution with a filesystem-level permission weakness (Hacking Articles, 2021).

#### Module 2e: Kernel CVE Scanner

The kernel scanner reads the current kernel version string from `/proc/version` and matches it against the CVE database in `kernel_cves.json`. Each entry in the database specifies a CVE identifier, a descriptive name, the affected kernel version range, a severity rating, a description of the vulnerability, and a reference URL. The module also performs a set of kernel hardening checks by reading `/proc/sys/kernel/` pseudo-files: it flags disabled ASLR (`randomize_va_space`), permissive ptrace scope (`yama/ptrace_scope`), and exposed kernel pointer values (`kptr_restrict`). The CVE database includes high-profile vulnerabilities such as CVE-2022-0847 (Dirty Pipe), CVE-2021-4034 (PwnKit), CVE-2021-3156 (Baron Samedit), and CVE-2016-5195 (Dirty COW).

#### Module 2f: Linux Capabilities Scanner

Linux capabilities provide a fine-grained alternative to the all-or-nothing SUID bit, dividing root privileges into distinct units that can be assigned to individual binaries (Kerrisk, 2010). However, certain capabilities are effectively equivalent to root access: `cap_setuid` allows spawning processes as any UID; `cap_sys_admin` enables a broad range of privileged operations; `cap_dac_override` bypasses file permission checks entirely. The capabilities scanner invokes `getcap -r / 2>/dev/null` and parses the output, cross-referencing binary names against `gtfobins.json` to identify those with known capability-based exploit paths. Binaries carrying dangerous capabilities are rated CRITICAL; those with less dangerous capabilities are rated HIGH.

#### Module 2g: Credential Scanner

The credential scanner searches for secrets that have been inadvertently stored in plaintext across a set of high-value locations. It reads shell history files (`.bash_history`, `.zsh_history`) for patterns matching passwords, API keys, and SSH passphrase arguments. It scans common configuration file locations for regex patterns matching AWS access keys, generic API tokens,

hardcoded password assignments, and private key headers. It checks for unprotected SSH private key files. It inspects files with extensions `.env`, `.conf`, `.cfg`, `.ini`, and `.xml` across `/etc/` for password-bearing lines. Shadow and group-shadow file readability is also assessed. Credential findings are treated with high severity given that exposed secrets often provide a direct path to further compromise without requiring any technical exploitation.

## Module 2h: PATH Hijacking Scanner

PATH hijacking occurs when an attacker can place a malicious binary in a directory that appears in the system PATH before the directory containing the intended binary. The PATH scanner checks for three conditions: (a) the presence of a dot (`.`) or empty string in the PATH, meaning the current directory is searched; (b) PATH directories that are writable by the current user; and (c) PATH directories that do not exist, into which an attacker could create the directory and populate it with malicious binaries. Additionally, the module parses systemd service unit files for ExecStart directives that use relative binary names rather than absolute paths, which makes service execution dependent on the service's PATH environment variable — a potential hijacking vector if that environment variable is attacker-influenced.

## Analysis Engine

### Deduplication and Severity Normalisation

The analysis engine, implemented in `analysis/engine.py`, receives all findings from all modules as a flat list of Python dictionaries. The first processing step normalises severity strings to one of four standard levels: CRITICAL, HIGH, MEDIUM, or LOW. The second step deduplicates findings using a composite key of (category, type, path), ensuring that the same binary or file reported by multiple code paths appears only once in the final output.

### Risk Scoring

After deduplication, findings are sorted in descending severity order using a four-level rank (CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1), with ties broken by category order and path. A weighted composite risk score is then computed as follows:

$$\text{Risk Score} = (\text{CRITICAL} \times 10) + (\text{HIGH} \times 5) + (\text{MEDIUM} \times 2) + (\text{LOW} \times 1)$$

This weighting reflects the dramatically higher exploitability of critical-severity findings. The overall risk level is assigned based on the presence of any CRITICAL finding or on score thresholds: scores of 20 or above map to CRITICAL; 10 or above map to HIGH; 5 or above map to MEDIUM; below 5 map to LOW. In practice, the presence of even a single CRITICAL

finding immediately elevates the overall risk to CRITICAL, drawing attention to the most urgent remediation priorities.

## Report Generation

### Text and JSON Reports

The text report generator, implemented in `reporter/generator.py`, produces a structured plain-text document with an executive summary section and a numbered findings section. When writing to a terminal, ANSI colour codes are applied to severity badges; when writing to a file, colour is suppressed for clean output. Each finding entry includes the severity badge, a sequential finding number, the category and type, the path or target, descriptive context, an exploit example (for educational reference), and a specific mitigation recommendation. The JSON report serialises the same data to a machine-readable format suitable for ingestion by security information and event management (SIEM) systems, vulnerability management platforms, or custom downstream tooling.

### HTML Report

The HTML report generator, implemented in `reporter/html_generator.py`, produces a fully self-contained single-file dashboard. All CSS and JavaScript are inlined — no external resources or Content Delivery Network (CDN) references are used, ensuring the report remains functional in air-gapped or offline environments. The dashboard presents a dark-themed interface with four summary count cards (CRITICAL, HIGH, MEDIUM, LOW), a system information table, a per-category breakdown table, and an expandable finding card for every individual finding. CRITICAL-severity findings are auto-expanded on page load to draw immediate attention. A print-friendly CSS media query allows the report to be rendered as a clean PDF directly from the browser.

## Implementation Results: Live Scan on Kali GNU/Linux 2025.2

### Test Environment

A live scan was conducted on a Kali GNU/Linux Rolling 2025.2 system with the following characteristics: hostname `attacker`, kernel version `6.12.25-amd64` (Kali 6.12.25-1kali1), current user `root` (`uid=0`, `gid=0`), `sudo` version `1.9.16p2`, and three users with active shell access (`root`, `postgres`, and `attacker`). The scan was run with default settings (full filesystem sweep, HTML output format) and completed successfully, producing the report file `PrivEsc Report`

— attacker — 2026-04-12 02\_06\_47.html in the output/directory. Running as root granted the scanner full read access to all files, including /etc/shadow, ensuring maximum coverage.

### Summary of Findings

The scan produced 114 total findings across five categories. Table 1 presents the distribution by category and severity. The overall risk level was assessed as CRITICAL.

Table 1

#### Distribution of Findings by Category and Severity

| Category              | Critical | High | Medium | Low | Total |
|-----------------------|----------|------|--------|-----|-------|
| SUID/SGID Binary      | —        | —    | —      | 36  | 36    |
| Misconfigured Service | —        | 2    | —      | —   | 2     |
| Kernel Security       | —        | 1    | 1      | 1   | 3     |
| Credentials           | 6        | 2    | —      | —   | 8     |
| PATH Hijacking        | —        | —    | 65     | —   | 65    |
| Total                 | 6        | 5    | 66     | 37  | 114   |

Note. The risk score computed by the analysis engine was  $(6 \times 10) + (5 \times 5) + (66 \times 2) + (37 \times 1) = 60 + 25 + 132 + 37 = 254$ . The presence of six CRITICAL findings independently elevated the overall risk level to CRITICAL.

### Critical Findings: Credential Exposure

All six CRITICAL findings belonged to the Credentials category and were classified as "Hardcoded Credential in Config." The credential scanner identified password-bearing lines in the following world-readable configuration files: /etc/cloud/cloud.cfg.d/20\_kali.cfg (containing a NOPASSWD:ALL sudo grant), /etc/legion.conf (containing a plaintext default password field),

`/etc/nsswitch.conf` (incorrectly flagged due to the word "passwd" in a configuration directive — a false positive), `/etc/redsocks.conf` (containing a commented and active plaintext password), `/etc/samba/smb.conf` (containing a passwd chat configuration string), and `/etc/unicomscan/modules.conf` (containing a database connection string with a plaintext password). All six files carried world-readable permissions (mode 0o644), meaning any user on the system could read them. The most actionable findings were `/etc/unicomscan/modules.conf` (a real database credential) and `/etc/redsocks.conf` (a real service password).

Recommended mitigations include replacing hardcoded credentials with environment variable references or a secrets management solution, and restricting file permissions to 0o640 with appropriate group ownership so that only authorised system accounts can read sensitive configuration.

### High Findings

The five HIGH-severity findings spanned three categories. Two Misconfigured Service findings identified systemd service unit files referencing binaries that did not exist on disk: `rc-local.service` referenced `/etc/rc.local`, and `quotaon-root.service` referenced `/usr/sbin/quotaon`. Both services are configured to run as root, meaning an attacker who could create the missing binary at the expected path would gain code execution as root on service start — a classic binary planting attack. The recommended remediation is either to create the binary with correct ownership and permissions or to disable the service entirely using `systemctl disable`.

One Kernel Security HIGH finding identified that the Yama Linux Security Module's `ptrace` scope restriction was set to 0 in `/proc/sys/kernel/yama/ptrace_scope`. A `ptrace` scope of 0 allows any process to attach to and inspect any other process owned by the same user, enabling memory reading and code injection. The recommended setting is 1 (restricted) or 2 (admin only). Two HIGH-severity Credential findings flagged that both `/etc/shadow` and `/etc/gshadow` were group-readable (mode 0o640), meaning members of the shadow group could read password hashes and attempt offline cracking. The recommended remediation is to confirm the group is set to shadow and that only trusted system processes belong to that group.

### Medium Findings: PATH Hijacking via Relative ExecStart

Sixty-five of the 66 MEDIUM findings were PATH Hijacking issues classified as "Relative ExecStart in Service File." The scanner identified systemd unit files in `/lib/systemd/system/` and `/etc/systemd/system/` whose ExecStart directives referenced binary names

without absolute paths. Representative examples include `getty-static.service` (referencing `systemctl`), `grub-common.service` (referencing `grub-editenv`), `faraday.service` (referencing `faraday-server`), `sshd-keygen.service` (referencing `ssh-keygen`), and `man-db.service` (referencing `+usr/bin/install` — the leading `+` prefix causes `systemd` to run the command with elevated privileges). When a service uses a relative binary name, resolution depends on the service's `Environment=PATH` value. If that value can be influenced by an attacker, they can substitute a malicious binary. The recommended mitigation for all 65 findings is to use fully qualified absolute paths in `ExecStart` directives.

One additional MEDIUM finding flagged that `/proc/sys/kernel/kptr_restrict` was set to 0, exposing raw kernel pointer addresses in `/proc` files and thereby assisting exploit development by providing memory layout information that would otherwise require bypassing kernel address space layout randomisation (KASLR).

#### Low Findings: SUID/SGID Binaries

All 36 LOW-severity findings were SUID/SGID binaries with no corresponding `GTFOBins` entry. These represent binaries that carry elevated permission bits for legitimate system functions — such as `passwd`, `ping`, and `su` — where no publicly documented exploitation technique exists. They are reported for completeness and to allow administrators to audit whether each binary's elevated permissions are still operationally required. One LOW Kernel Security finding noted that ASLR was enabled and properly configured on the test system, confirming that basic kernel memory randomisation was in place.

#### Discussion

##### Effectiveness of Automated Detection

The live scan demonstrated that automated detection can identify real, actionable vulnerabilities that might be overlooked during manual auditing. The discovery of plaintext database credentials in `/etc/unicornscan/modules.conf` and a live service password in `/etc/redssocks.conf`, both in world-readable files, is precisely the kind of finding that is easy to miss during a manual review of hundreds of configuration files. The missing service binary findings for `rc-local.service` and `quotaon-root.service` represent a genuine privilege escalation risk that requires only the ability to create a file at a known path — no memory corruption or complex exploit chain required.

The volume of PATH Hijacking medium findings (65 out of 114 total) illustrates a known challenge with automated scanning: high false-positive rates in categories where the theoretical risk exists but practical exploitability is context-dependent. Most of the flagged service files are part of the standard Kali Linux distribution and are unlikely to be exploitable in practice, because the system PATH for root-run services is tightly controlled. This underscores the importance of treating automated scanner output as a starting point for human review rather than an authoritative list of exploitable vulnerabilities (Shu et al., 2017).

### False Positives and Limitations

Several of the credential findings warrant closer examination as potential false positives. The `/etc/nsswitch.conf` finding was triggered by the word "passwd" appearing in a legitimate Name Service Switch configuration directive, not as a credential. Similarly, the `/etc/samba/smb.conf` finding was triggered by a `passwd` chat directive that is part of the standard Samba password change protocol, not a hardcoded secret. These false positives highlight a limitation of pattern-based credential detection: regular expressions that match common password-adjacent words will inevitably produce noise in configuration files that use those words as directive names rather than secret values. Future versions could reduce false positives through contextual analysis — for example, by checking whether the match appears on the right-hand side of an assignment operator with a non-empty, non-placeholder value.

The tool's detection coverage is bounded by what is enumerable at runtime with the scanning user's privileges. While running as root provides comprehensive access in this test, non-root scans will miss checks that require reading `/etc/shadow`, running `getcap` without elevated permissions, or accessing other users' home directories. Additionally, kernel CVE matching is version-string-based rather than patch-state-based: a distribution that backports a security fix without changing the kernel version string would be incorrectly flagged as vulnerable. This is a known limitation of all version-comparison-based scanners (Cvedetails.com, 2023).

### Ethical Considerations

PRIVESC was designed with a strong ethical framework. The tool is detection-only and includes no exploitation code that could cause harm on a target system. Exploit examples included in reports are clearly labelled for educational reference only. The project is accompanied by a comprehensive `DISCLAIMER.md` document that specifies authorised use cases (owned systems, written- permission engagements, CTF competitions, educational lab

environments), lists the criminal statutes that prohibit unauthorised use in major jurisdictions (including the Computer Fraud and Abuse Act in the United States and the Computer Misuse Act 1990 in the United Kingdom), and outlines responsible disclosure procedures for any vulnerabilities discovered. This ethical framing is essential for any tool distributed in a security education context (Holt & Bossler, 2016).

## Conclusion

The Linux Privilege Escalation Automation Toolkit delivers on its core objectives: it provides a clean, readable, modular Python implementation of the most common Linux privilege escalation checks, pairs them with a principled risk-scoring engine, and outputs findings in formats suitable for both human review and machine consumption. The live scan on a Kali GNU/Linux 2025.2 system identified 114 findings including six critical-severity credential exposure issues and two high-severity binary planting vulnerabilities, demonstrating that the tool surfaces real, actionable security weaknesses in a realistic environment.

The project makes a meaningful contribution to security education by providing a codebase that is intentionally readable and well-documented — every module can be studied in isolation to understand the underlying vulnerability class it targets. It mirrors industry-standard tools like LinPEAS in functional scope while offering a Python-native implementation that is easier to extend, test, and present in an academic context.

Future directions include reducing false positives in the credential scanner through contextual analysis, adding patch-level CVE matching to distinguish backported fixes from vulnerable kernels, expanding the GTFOBins and kernel CVE databases, implementing a comparison mode that diffs two scan results to track remediation progress over time, and adding a structured recommendations report that groups mitigations by effort level to support remediation prioritisation.

The toolkit demonstrates that automated security enumeration, when designed with clarity, modularity, and ethical constraints, can be both a practical operational tool and an effective vehicle for teaching the principles of Linux security hardening and privilege escalation defence.



## References

- Andress, J., & Winterfeld, S. (2014). *Cyber warfare: Techniques, tactics and tools for security practitioners* (2nd ed.). Syngress. <https://doi.org/10.1016/B978-0-12-416672-1.00001-5>
- Cvedetails.com. (2023). *CVE security vulnerability database*. CVE Details. <https://www.cvedetails.com/>
- Hacking Articles. (2021). *Linux privilege escalation using PATH variable*. Hacking Articles. <https://www.hackingarticles.in/linux-privilege-escalation-using-path-variable/>
- Holt, T. J., & Bossler, A. M. (2016). *Cybercrime in progress: Theory and prevention of technology-enabled offenses*. Routledge.
- Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.
- MITRE Corporation. (2023). *ATT&CK for Enterprise: Privilege escalation (TA0004)*. MITRE ATT&CK. <https://attack.mitre.org/tactics/TA0004/>
- Polop, C. (2021). *PEASS-ng: Privilege escalation awesome scripts suite* [Software]. GitHub. <https://github.com/carlospolop/PEASS-ng>
- Rebootuser. (2019). *LinEnum: Scripted local Linux enumeration and privilege escalation checks* [Software]. GitHub. <https://github.com/rebootuser/LinEnum>
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Shu, X., Tian, K., Ciambrone, A., & Yao, D. (2017). Breaking the target: An analysis of target data breach and lessons learned. *arXiv preprint*. <https://arxiv.org/abs/1701.04940>
- Tipton, H. F., & Krause, M. (Eds.). (2007). *Information security management handbook* (6th ed.). Auerbach Publications.
- Wagner, N. (2018). *BeRoot: Privilege escalation project — Windows / Linux / Mac* [Software]. GitHub. <https://github.com/AlessandroZ/BeRoot>
- Wildner, F., & Polop, C. (2019). *GTFOBins: Curated list of Unix binaries that can bypass local security restrictions in misconfigured systems*. GTFOBins. <https://gtfobins.github.io/>